

ROSE——编译原理逆向工程工具

实验二-设计文档

学　　号: 23336179

姓　　名: 马福泉

专　　业: 计算机科学与技术

指 导 教 师: 李文军

起 始 日 期: 2026 年 6 月

结 束 日 期: 2026 年 6 月

实 验 地 点: B203

目录

1 实验简述 **2**

2 Oberon-0 词汇表 **2**

3 正则定义式 **2**

4 词法规则与 C/C++/Java 的异同比较 **3**

5 JFlex、JLex、GNU Flex 工具对比 **4**

6 JFlex 生成与编译结果 **5**

7 变异测试验证 **6**

7.1 Javadoc 设计文档生成 9

8 实验心得与体会 **10**

1 实验简述

本项目 ROSE 是一个编译原理逆向工程工具，旨在对 Oberon-0 语言源程序进行词法分析、语法分析、语义分析，并自动生成流程图和调用关系图。本实验为第二阶段——使用 JFlex 工具自动生成词法分析器 OberonScanner，实现对 Oberon-0 源程序的词法单元识别与错误检测。

2 Oberon-0 词汇表

Oberon-0 的词汇体系可划分为保留字、预定义关键字、运算符与分界符、标识符、字面量及注释六大类别。表 1 列出了完整的词汇表。

表 1: Oberon-0 完整词汇表

类别	词汇
保留字	MODULE, BEGIN, END, IF, THEN, ELSIF, ELSE, WHILE, DO PROCEDURE, VAR, CONST, TYPE, RECORD, ARRAY, OF DIV, MOD, OR, ~
预定义关键字	INTEGER, BOOLEAN, TRUE, FALSE, Read, Write, WriteLn
运算符与分界符	+, -, *, DIV, MOD　（算术） =, #, <, <=, >, >=　（关系） &, OR, ~　（逻辑） :=, :, ;, . , ,　（赋值与分界） (,), [,]　（括号）
标识符	{Letter}({Letter}|{Digit})*, 长度 ≤ 24
字面量	十进制整数: 0 | [1-9]{Digit}* 八进制整数: 0[0-7]+
注释	(* ... *)　（不可嵌套）

3 正则定义式

以下以纯文本正则定义式描述 Oberon-0 的词法规则（使用 $\rightarrow$ 表示产生、ϵ 表示空串）：

- **基本符号**

```
digit  -> 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
letter -> a | b | ... | z | A | B | ... | Z
```

- **空白字符**

```
whitespace -> ' ' | '\t' | '\r' | '\n'
```

- **标识符**

```
identifier -> letter (letter | digit)*
```

限制：生成的标识符长度必须 ≤ 24。

- **整数**

```
decInteger -> 0 | [1-9] digit*
octInteger -> 0 (0 | 1 | ... | 7)+
```

约束：(1) 十进制整数首位非零（除非值为 0）；(2) 八进制整数首位必须为 0，且每位 $\in$ [0-7]。

- **注释**

```
comment     -> "(*" (^("*)"))* "*)"
```

其中 `^("*)")` 表示除"*)" 外的任意字符序列（不支持嵌套）。

- **保留字集合 (Keyword Set K)**

```
K -> { MODULE, BEGIN, END, IF, THEN, ELSIF, ELSE,
       WHILE, DO, PROCEDURE, VAR, CONST, TYPE,
       RECORD, ARRAY, OF, DIV, MOD, OR, ~ }
```

4 词法规则与 C/C++/Java 的异同比较

Oberon-0 的词法规范与 C/C++/Java 在多个维度上存在显著差异，表 2 给出系统对比。

表 2: Oberon-0 与 C/C++/Java 词法差异对比

维度	Oberon-0	C/C++/Java
大小写敏感性	大小写无关 (case-insensitive)；所有保留字、标识符忽略大小写	大小写敏感；`int` 与 `Int` 是不同的标识符
标识符长度限制	硬限制 24 字符；超出触发异常	C99 $\leq$ 31（内部），Java 无硬性上限
注释语法	仅支持 `(* ... *)`，不可嵌套	`//` 单行注释、`/* ... */` 多行注释
整数字面量	十进制 + 八进制 (`0` 前缀)；八进制含 8/9 触发异常	C/Java 支持十进制/八进制 (`0`)/十六进制 (`0x`)
数字-标识符边界	强制要求空格分隔；`123abc` 触发异常	C/Java 词法规则中数字后接字母自动作为两个 token
运算符符号	`DIV`, `MOD`, `&`, `OR`, `~`, `:=`, `#`	`/`, `%`, `&&`, `||`, `!`, `=`, `!=`

Oberon-0 的关键设计差异可归纳为：

1. **大小写无关**：这是 Pascal 系语言的共同特征，简化了 DFA 状态数量（无需为 `A-Z` 和 `a-z` 分别维护匹配分支）。JFlex 通过 `%ignorecase` 指令一行配置即可生效。
2. **标识符长度硬约束**：Oberon-0 的 24 字符上限并非 JFlex 声明式规则所能表达，必须在词法动作中以 Java 代码手动检查长度并在超出时抛出 `IllegalIdentifierLengthException`。
3. **数字-标识符边界强制分离**：C 语言将 `123abc` 自动拆分为两个 token（`123` 和 `abc`），而 Oberon-0 将其视为词法错误。这是通过正则 `{Digit}+{Letter}+` 并关联异常抛出实现的——本质上是一种" 先匹配后拒绝" 的策略。
4. **八进制整数的严格检查**：Oberon-0 仅接受 `0[0-7]+` 作为有效八进制，`08` 或 `019` 需被独立的正则模式 `0[0-9]*[89]+[0-9]*` 捕获并抛出 `IllegalOctalException`。
5. **注释不支持嵌套**：简化了词法分析器的状态转移逻辑——一个 `COMMENT` 状态即可处理，无需维护嵌套深度计数器。

5 JFlex、JLex、GNU Flex 工具对比

表 3 从生成语言、算法、性能和功能四个维度对比三种主流词法分析器生成工具。

表 3: JFlex vs JLex vs GNU Flex 对比

维度	**GNU Flex**	**JLex**	**JFlex**
目标语言	C / C++	Java	Java
首次发布	1987 (继承自 Lex)	1996	1998
DFA 生成算法	表驱动 DFA	表驱动 DFA	NFA → DFA 优化；行压缩表（row-displaced table）
Unicode 支持	有限（依赖 C locale）	有限（Java 1.0 时代）	**完整 Unicode 支持** (基于 Java `char` 的 16-bit 码点以及代理对)
正则语法	POSIX 扩展正则	基础正则	扩展正则 + 宏定义 + **先行断言** + `%state` 状态栈
性能	极高 (C 原生)	中等	高（相比 JLex 提升 2–3 倍扫描速度）
与 CUP 集成	不适用	支持	**原生支持 CUP** (`%cup` 模式自动生成 `Symbol` Token)
维护状态	活跃 (GNU 项目)	停止维护 (1998)	**活跃维护** (最新版 1.9+, 2023)

GNU Flex 是 Lex 的 GNU 替代品，主导了 C/C++ 生态的词法分析器生成长达三十余年。其生成的扫描器以 C 语言原生性能著称，但在 Unicode 处理和大字符集场景中表现受限，且不适合 Java 工程集成。

JLex 是普林斯顿大学 Elliot Berk 于 1996 年发布的 Java 版 Lex 移植，首次将 Lex 理念引入 JVM 生态。然而其 DFA 生成算法沿用经典的子集构造法，未做进一步优化；Unicode 支持仅限于 Java 1.0 时期的 `char` 范围；项目自 1998 年起已停止维护。

JFlex 由 Gerwin Klein 基于 JLex 完全重写，现已成为 Java 生态词法分析器生成的事实标准。其核心优势包括：(1) 更优的 DFA 压缩算法（行列位移表），生成的分析器性能比 JLex 快 2–3 倍；(2) 完整的 Unicode 支持，允许在正则中直接使用 `\uXXXX` 和 Unicode 类别表达式；(3) 丰富的声明式指令——`%ignorecase`、`%state`（独占式/包容式状态）、宏系统——使得词法规则的定义高度模块化；(4) 与 CUP（Java 版 Yacc）的深度集成，`%cup` 模式可直接生成与语法分析器对接的 `Symbol` Token 流，无需手动编写胶水代码。

本实验选择 JFlex 作为词法分析器生成工具，既因其作为 Java 生态最成熟的 Lex 变体，能无缝融入 ROSE 工具链的 Java 技术栈；也因其丰富的状态管理能力和异常检测机制（通过词法动作中的 Java 代码注入）恰好满足 Oberon-0 对标识符长度、数字-标识符边界、八进制合法性等细粒度错误检测的特殊需求。

6　JFlex 生成与编译结果

```
(base) PS E:\3.2\Compilers\LAB\my_lab\lab5\ex2> .\gen.bat
[gen] JFLEX_HOME=E:\3.2\Compilers\LAB\my_lab\lab5\ex2\jflex
[gen] 正在调用 JFlex 生成词法分析器 ...
Reading "src\oberon.flex"
Constructing NFA : 256 states in NFA
Converting NFA to DFA :
..........................................................................
.....................................
167 states before minimization, 159 states in minimized DFA
Old file "src\OberonScanner.java" saved as "src\OberonScanner.java~"
Writing code to "src\OberonScanner.java"

[gen] OberonScanner.java 已生成至 src/ 目录。
[gen] 下一步：运行 build.bat 编译。
Press any key to continue . . .
```

图 1: gen.bat 执行结果：256 NFA 状态 → 159 最小化 DFA 状态

```
(base) PS E:\3.2\Compilers\LAB\my_lab\lab5\ex2> .\build.bat
[build] 正在编译 Java 源文件 ...
[build] 编译成功，class 文件已输出至 bin/ 目录。
Press any key to continue . . .
(base) PS E:\3.2\Compilers\LAB\my_lab\lab5\ex2>
```

图 2: build.bat 执行结果：Token.java + exceptions + OberonScanner.java 全部编译通过

7　变异测试验证

使用实验一中构造的 8 个变异文件对 OberonScanner 进行错误检测能力验证。图 3 至图 10 展示了各变异文件的扫描结果——词法级别的 4 种错误全部被正确捕获并抛出对应异常，语法和语义级别的错误在词法阶段正常通过（留待后续实验处理）。

```
(base) PS E:\3.2\Compilers\LAB\my_lab\lab5\ex2> .\test.bat ..\ex1\testcases\mut1_IllegalSymbol.obr
[test] 正在运行测试: ..\ex1\testcases\mut1_IllegalSymbol.obr
(3,1) PROCEDURE
(3,11) IDENT [CalcTotalAndAvg]
(3,26) (
(3,27) VAR
(3,31) IDENT [s]
(3,33) :
(3,35) IDENT [Student]
(3,42) )
(3,43) ;
(4,1) VAR
(5,3) IDENT [sum]
(5,7) :
(5,9) INTEGER_TYPE
(5,16) ;
(6,3) IDENT [k]
(6,7) :
(6,9) INTEGER_TYPE
(6,16) ;
(7,1) BEGIN
(8,3) IDENT [sum]
(8,7) :=
(8,10) INTEGER [0]
(8,11) ;
(9,3) IDENT [k]
(9,5) :=
(9,8) INTEGER [0]
(9,9) ;
(10,3) WHILE
(10,9) IDENT [k]
(10,11) <
(10,13) IDENT [CourseCount]
(10,25) DO
(11,5) IDENT [sum]
(11,9) :=
(11,12) IDENT [sum]
[错误] IllegalSymbolException
  非法符号: '@' (0x40)
Press any key to continue . . .
```

图 3: 变异 1: IllegalSymbol ——非法符号 @ 被正确捕获

```
(base) PS E:\3.2\Compilers\LAB\my_lab\lab5\ex2> .\test.bat ..\ex1\testcases\mut2_IllegalInteger.obr .\test.bat ..\ex1\testc
ses\mut2_IllegalInteger.obr
(10,20) ]
(10,21) )
(10,22) ;
(11,5) IDENT [k]
(11,7) :=
(11,10) IDENT [k]
(11,12) +
(11,14) INTEGER [1]
(12,3) END
(12,6) ;
(13,3) IDENT [CalcTotalAndAvg]
(13,18) (
(13,19) IDENT [s]
(13,20) )
(13,21) ;
(15,3) IDENT [k]
(15,5) :=
[错误] IllegalIntegerException
  数字与标识符粘连: "123k"
Press any key to continue . . .
```

图 4: 变异 2: IllegalInteger ——数字标识符粘连 123k 被正确捕获

```
(base) PS E:\3.2\Compilers\LAB\my_lab\lab5\ex2> .\test.bat ..\ex1\testcases\mut3_IllegalOctal.obr
(10,3) IDENT [best]
(10,8) :=
(10,11) IDENT [t]
(10,12) [
(10,13) INTEGER [0]
(10,14) ]
(10,15) ;
(11,3) IDENT [maxTot]
(11,10) :=
(11,13) IDENT [t]
(11,14) [
(11,15) INTEGER [0]
(11,16) ]
(11,17) .
(11,18) IDENT [Total]
(11,23) ;
(12,3) IDENT [idx]
(12,7) :=
[错误] IllegalOctalException
  非法八进制字面量: "08"
Press any key to continue . . .
```

图 5: 变异 3: IllegalOctal ——非法八进制 08 被正确捕获

```
(base) PS E:\3.2\Compilers\LAB\my_lab\lab5\ex2> .\test.bat ..\ex1\testcases\mut4_MismatchedComment.obr
(4,1) VAR
(5,3) IDENT [sum]
(5,7) :
(5,9) INTEGER_TYPE
(5,16) ;
(6,3) IDENT [k]
(6,7) :
(6,9) INTEGER_TYPE
(6,16) ;
(7,1) BEGIN
(8,3) IDENT [sum]
(8,7) :=
(8,10) INTEGER [0]
(8,11) ;
(9,3) IDENT [k]
(9,5) :=
(9,8) INTEGER [0]
(9,9) ;
[错误] MismatchedCommentException
  注释不闭合: 缺少 '*)'
Press any key to continue . . .
```

图 6: 变异 4: MismatchedComment ——注释不闭合被正确捕获

```
(base) PS E:\3.2\Compilers\LAB\my_lab\lab5\ex2> .\test.bat ..\ex1\testcases\mut5_MissingParenthesis.obr
(16,7) IDENT [ok]
(16,10) :=
(16,13) BOOL_LIT [false]
(17,5) END
(17,8) ;
(18,5) IDENT [idx]
(18,9) :=
(18,12) IDENT [idx]
(18,16) +
(18,18) INTEGER [1]
(19,3) END
(19,6) ;
(20,3) IDENT [result]
(20,10) :=
(20,13) IDENT [ok]
(21,1) END
(21,5) IDENT [CheckAllPass]
(21,17) ;

扫描完成: 82 个 Token
Press any key to continue . . .
```

图 7: 变异 5: MissingParenthesis ——语法错误，词法阶段正常通过

```
(base) PS E:\3.2\Compilers\LAB\my_lab\lab5\ex2> .\test.bat ..\ex1\testcases\mut6_MissingOperator.obr
(13,3) END
(13,6) ;
(14,3) IDENT [s]
(14,4) .
(14,5) IDENT [Total]
(14,11) :=
(14,14) IDENT [sum]
(14,17) ;
(15,3) IDENT [s]
(15,4) .
(15,5) IDENT [Average]
(15,13) :=
(15,16) IDENT [sum]
(15,20) DIV
(15,24) IDENT [CourseCount]
(16,1) END
(16,5) IDENT [CalcTotalAndAvg]
(16,20) ;

扫描完成: 65 个 Token
Press any key to continue . . .
```

图 8: 变异 6: MissingOperator ——语法错误，词法阶段正常通过

```
(base) PS E:\3.2\Compilers\LAB\my_lab\lab5\ex2> .\test.bat ..\ex1\testcases\mut7_TypeMismatched.obr
(25,3) IF
(25,6) IDENT [PassAll]
(25,14) THEN
(26,5) WRITE
(26,10) (
(26,11) INTEGER [1]
(26,12) )
(27,3) ELSE
(28,5) WRITE
(28,10) (
(28,11) INTEGER [0]
(28,12) )
(29,3) END
(29,6) ;
(30,3) WRITELN
(31,1) END
(31,5) IDENT [PrintAll]
(31,13) ;

扫描完成: 110 个 Token
Press any key to continue . . .
```

图 9: 变异 7: TypeMismatched ——语义错误，词法阶段正常通过

```
(base) PS E:\3.2\Compilers\LAB\my_lab\lab5\ex2> .\test.bat ..\ex1\testcases\mut8_ParameterMismatched.obr
(10,21) )
(10,22) ;
(11,5) IDENT [k]
(11,7) :=
(11,10) IDENT [k]
(11,12) +
(11,14) INTEGER [1]
(12,3) END
(12,6) ;
(15,3) IDENT [CalcTotalAndAvg]
(15,18) (
(15,19) IDENT [s]
(15,20) ,
(15,22) INTEGER [0]
(15,23) )
(16,1) END
(16,5) IDENT [ReadStudent]
(16,16) ;

扫描完成: 57 个 Token
Press any key to continue . . .
```

图 10: 变异 8: ParameterMismatched ——语义错误，词法阶段正常通过

如上所示，变异 1–4 在词法分析阶段即被 OberonScanner 准确拦截并抛出对应异常，验证了词法分析器对 IllegalSymbol、IllegalInteger、IllegalOctal、MismatchedComment 四类错误的检测能力。变异 5–8（语法/语义层错误）在词法阶段正常通过，留待语法分析器和语义分析器进一步处理，体现了编译器前端分层过滤的工程架构。

7.1 Javadoc 设计文档生成

本实验额外提供了 `gendoc.bat` 脚本，调用 JDK 自带的 `javadoc` 工具对 `src/` 下所有手写 Java 源码（Token.java、OberonScannerTest.java 及 5 个自定义异常类）生成 HTML 格式的 API 设计文档，输出至 `doc/` 目录。自动生成的文档以标准 Javadoc 样式呈现类层次结构、字段说明和方法签名，便于后续维护与查阅。

8 实验心得与体会

JFlex 的宏系统（Macros）大幅提升了规则的模块化程度——先定义 `Letter`、`Digit`、`Identifier` 等基础宏，再在规则段以组合形式引用，使得整体规则清晰可读。而在遇到” 标识符长度 24” 和” 八进制非法数字” 这类无法用纯正则表达的约束时，词法动作中的 Java 代码注入展现了声明式框架与命令式编程协作的能力。

自定义异常体系（`IllegalSymbolException`、`IllegalIntegerException`、`IllegalOctalException`、`MismatchedCommentException`、`IllegalIdentifierLengthException`）的设计使我意识到：词法分析器不仅是” 将字符流切分为 Token 流” 的机械过程，更是一个**具有语义判断能力的前端过滤器**。通过在词法阶段即精确报告错误的行列号与具体原因，可以为后续语法分析和用户错误定位提供精准的上下文信息。

此外，对 JFlex/JLex/Flex 三种工具的对比分析让我理解了一个深刻的原则：**工具的选择决定了编译器的上限，但规则的设计决定了编译器的下限**。JFlex 凭借优化后的 DFA 压缩算法和完整的 Unicode 支持提供了性能与灵活性的保障，但若规则定义不当（如漏掉注释状态中对 EOF 的处理），仍会在边界 case 中产生漏检。